에세이3

2022012408
염재민

#1
우선 교수님의 지시대로 Q13, 14를 업데이트 진행하였고 Talk 디렉토리에서 인공지능과 대화한 내용을 검토해보았습니다. talks_1.txt, talks_2.txt, talks_3.txt 파일들을 검토진행하였으며, 대화 기록에서는 Space Invaders RTL 설계의 칩 면적과 논리회로 구성, 코드 수정 방향 등을 다루고있었습니다.
특히 변수 선언 누락, 스프라이트 배열 초기화, 반복문에서 발생할 수 있는 부호 없는 정수의 언더플로 문제를 확인하게됬습니다. 또한 반복 연산을 줄여 하드웨어 자원을 절약하기 위한 설계 변경 방안도 검토했습니다.

#2
다음으로는 `simulation` 디렉터리의 Makefile을 확인하고, 여러 버전의 Verilog RTL 코드가 어떻게 다른지 비교하였습니다.
Verilog RTL은 디지털 회로가 어떤 입력을 받아 어떤 동작을 수행하는지 표현한 설계 코드이며 같은 우주 침략자 게임을 구현하더라도 버튼 입력을 처리하는 방법이나 적과 플레이어의 충돌을 처리하는 방법 등에 따라 코드가 달라질 수 있는걸 알고 시작했습니다.

먼저 Makefile을 확인하여 여러 RTL 버전 중 하나를 선택하고, 선택한 코드를 이용해 시뮬레이션 프로그램을 빌드하고 실행하는 과정을 살펴보았고, 이후 버전 0부터 버전 4까지의 코드를 비교하여 주요 차이점을 확인하였고 아래에 정리를 해보았다.

1) 게임 상태를 관리하는 방법
게임 상태는 게임이 시작되기 전.중.후 상황을 구분하기 위한 것. RTL 버전에 따라 상태를 표현하는 방법과 상태가 바뀌는 과정이 달랐다. 일부 버전에서는 버튼 입력을 확인하는 상태를 추가하여 입력 처리 과정을 구분.

2) 버튼 입력 처리 방법
게임을 시작하거나 발사 버튼을 누르는 입력을 처리하는 방식이 달랐다. 일부 버전에서는
버튼의 이전 입력을 저장하여 버튼을 새롭게 눌렀을 때를 감지하는 방식을 사용하였다. 다른 버전에서는 버튼 입력 신호를 직접 사용하였다. 이러한 차이는 버튼 입력을 감지하고 처리하는 방식에 영향을 줌.

3) 충돌 처리 방법
플레이어와 적의 충돌을 처리하는 방식도 변경되었다. 일부 버전에서는 충돌 사실을 별도의 신호로 저장한 뒤 게임 상태를 변경하였다. 다른 버전에서는 충돌이 발생하면 즉시 게임 상태를 변경하였다. 충돌을 감지하는 시점과 게임 상태를 바꾸는 과정이 서로 다르다는 점을 확인하였다.

4) 게임 진행 속도 제어
게임의 진행 속도를 조절하기 위해 사용되는 카운터 값도 버전에 따라 달랐다. 버전 2에서 버전3으로 변경되는 과정에서는 게임 진행을 제어하는 상수 값이 `20'd833333`에서 `20'd16000`으로 변경되었다. 이 값은 게임 동작이 갱신되는 시점에 영향을 주므로, 실제 속도는 입력 클록과 카운터 동작을 함께 고려해야 한다.

5) 최종 버전의 수정 사항
버전 3에서 버전 4로 변경되면서 버튼 입력을 감지하는 처리와 충돌을 별도로 확인하는 방식이 다시 적용되었다. 또한 총알의 이동과 충돌 처리가 수정되었다. 이를 통해 RTL 코드는 단순히 기능을 추가하는 것뿐 아니라, 게임 동작이 의도대로 이루어지도록 처리 순서와 조건을 조정하면서 개선된다는 점을 확인하였다.

이렇게 버전 0부터 버전 4까지의 주요 코드 차이를 비교하였으며, 그 결과 게임 상태 관리, 버튼 입력 감지, 충돌 처리, 게임 진행 속도 제어 방식 등이 버전에 따라 달라지는 것을 확인하였다.
같은 게임을 구현한 RTL이라도 내부 동작을 처리하는 방식에 따라 코드 구조가 달라질 수 있음을 확인하였다. 따라서 하드웨어 설계에서는 단순히 코드가 작성되어 있는지만 확인하는 것이 아니라, 각 조건과 상태 변화가 실제 회로 동작에 어떤 영향을 주는지 이해하는 것이 중요하다는 생각이 들었다.

#3
다음으로는 보드를 배포받기전 FPGA를 이용해 설계한 하드웨어가 제대로 동작하는지 확인하기 위한 검증 환경을 준비하였다. 보드를 연결해 실행하는 대신 필요한 프로그램들을 빌드하는 작업을 진행하였음.

먼저 **Modeling Interface**는 컴퓨터와 하드웨어 사이에서 데이터를 주고받도록 도와주는 부분이다. Makefile을 확인한 결과, Raspberry Pi Pico 계열 보드를 대상으로 프로그램을 빌드하도록 설정되어 있어서 해당 보드에서 사용할 프로그램을 빌드하였으며, 처음에는 필요한 라이브러리가 없어 오류가 발생했지만 U8g2 라이브러리를 설치한 뒤 빌드에 성공하였다.

다음으로 **FPGA Transactor**를 빌드하였다. 이 과정에서는 Quartus를 이용해 FPGA에 사용할 설계 파일을 만들었다. 먼저 `.sof` 파일을 생성하고, 이를 FPGA 설정에 사용할 수 있는 `.rbf` 파일로 변환하였다. 두 과정 모두 정상적으로 완료되었다.

마지막으로 **SystemC 테스트벤치**를 빌드하였다. 테스트벤치는 설계한 하드웨어가 예상대로 동작하는지 확인하기 위한 시험 환경이다. 빌드 결과 실행 파일이 정상적으로 생성되었다. 다만 보드가 연결되지 않은 상태에서 실행했기 때문에 시리얼 포트 오류가 발생했으며, 실제 보드를 이용한 통신과 동작 검증은 진행을 해보아야 확실해질거같다.

#4
마지막으로 앞선 과제 2번속 Makefile에 각 단계마다 어떤 툴을 동원하여 명령을 주고있는지 정리해보려한다. 정리전 Makefile의 역할을 한번더 정리하면 프로그램을 빌드하고 실행하는 과정을 명령어로 정리해 놓은 파일이다.
** RTL 버전 선택 단계**
Makefile에서는 `ln -s` 명령어를 사용하여 여러 Verilog RTL 파일 중 선택한 버전을 `space_invaders_engine_glcd.v`라는 이름으로 연결한다. 심볼릭 링크는 실제 파일을 복사하지 않고 특정 파일을 가리키도록 만드는 기능이다. 이를 통해 Makefile을 수정하지 않고도 사용할 RTL 버전을 바꿀 수 있다.

** 코드 검사 단계 (`lint`)**
`verilator --lint-only` 계열 명령어를 사용한다. Verilator는 Verilog RTL의 문법과 설계상 문제가 될 수 있는 부분을 검사하는 도구이다. 이를 통해 실제 빌드를 진행하기 전에 코드의 오류나 경고를 확인할 수 있다.

** 빌드 단계 (`build`)**
`verilator --sc --trace --timing --pins-sc-uint --top-module ... --exe --build` 명령어를 사용한다. Verilator는 Verilog RTL을 SystemC 환경에서 사용할 수 있도록 변환하고, 테스트벤치와 함께 실행 프로그램을 빌드한다. 이 과정에서 Clang++ 컴파일러와 SystemC 및 SDL2 라이브러리가 사용된다.

** 실행 단계 (`run`)**
`./obj_dir/Vspace_invaders_engine_glcd` 명령어로 빌드 결과 생성된 실행 파일을 실행한다. 이를 통해 게임 화면이 나타나는지, 버튼 입력에 따라 게임이 동작하는지 확인할 수 있다.

** 파형 확인 단계 (`wave`)**
`gtkwave` 명령어로 시뮬레이션 결과가 저장된 VCD 파일을 연다. GTKWave는 시간에 따라 신호가 어떻게 변하는지 파형으로 보여 주므로, 게임 상태나 버튼 입력 등 내부 신호의 변화를 확인하는 데 사용된다.

** 정리 단계 (`clean`)**
`rm` 등의 셸 명령어를 사용하여 빌드 과정에서 생성된 실행 파일과 임시 결과물을 삭제한다. 이를 통해 불필요한 파일을 정리할 수 있다.

이처럼 Makefile은 각 단계에서 서로 다른 도구와 명령어를 사용하여 RTL 선택, 코드 검사, 빌드, 실행 및 파형 확인 과정을 관리한다. 각 도구의 역할을 이해하면 단순히 `make` 명령을 실행하는 것을 넘어, 하드웨어 설계 코드가 검사되고 실행되는 과정을 이해할 수 있음을 알게 되었습니다.